

Projects Models

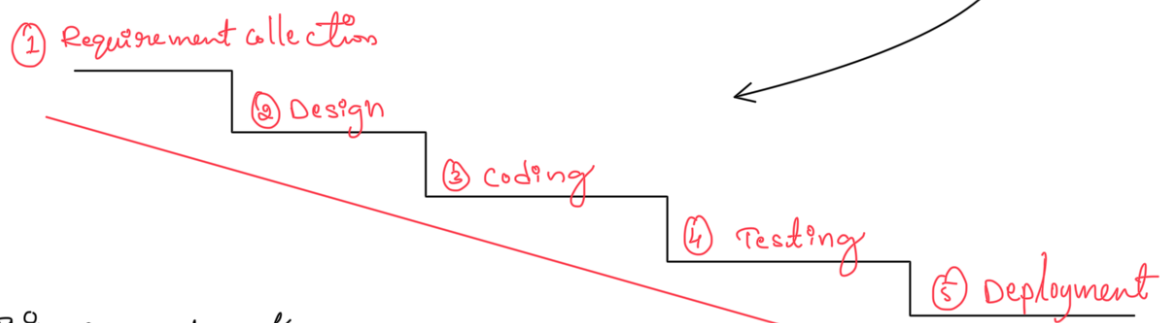
DAY 100
18 / 10 / 2025

A project Have Multiple phases

- ① Requirement collection
- ② Design (System Design, Backend design, User Interface design)
- ③ Coding
- ④ Testing
- ⑤ Deployment / shipping / Rollout

We can broadly classify Entire Activity into 5 phases

Traditionally Industry follow something called as Waterfall Model

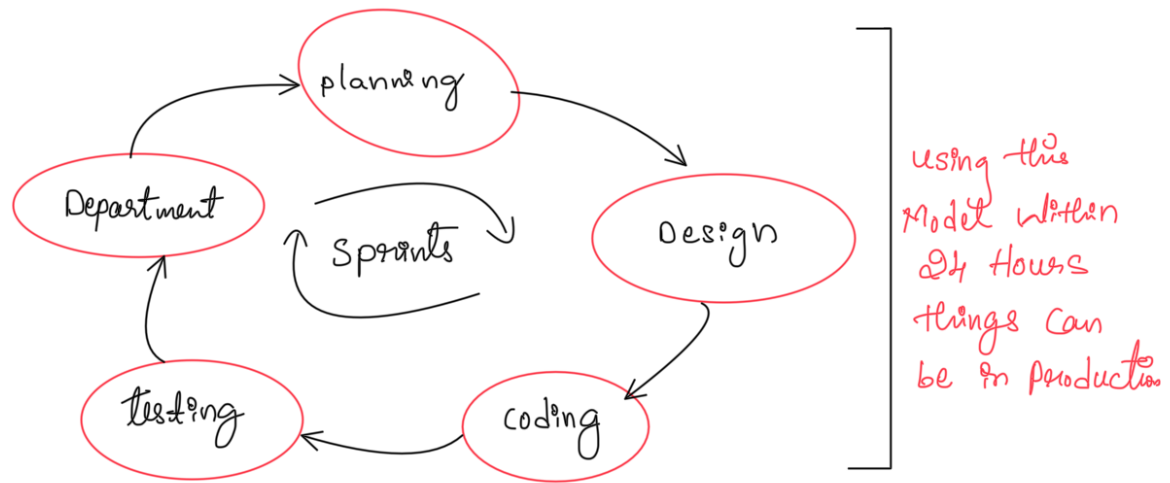


This Approach of
Moving one phase to other
is called as Waterfall Model

Waterfall Model takes a very long time from Requirement collection to Deployment through step by step process. Any changes on project takes long time.

So Another Model called Agile Model was introduced which helps to try different features, run the experiments to see which work which wouldn't work

Agile Model is like a cycle



💧 Waterfall Model

The **Waterfall Model** is the **oldest and simplest** SDLC model.

It's called "Waterfall" because progress flows **downward like a waterfall** — one phase finishes completely before the next begins.

◆ Structure:

📄 Copy

Requirements → Design → Coding → Testing → Deployment → Maintenance

🧩 Phases in a Software Project

Every software project, no matter the size, goes through **five fundamental stages**:

Phase	Description	Example
1 Requirement Collection	Understand what the customer or user wants.	"App should track expenses and show monthly reports."
2 Design	Create blueprints — architecture, database, UI/UX, data flow.	"Use Java backend + MySQL + REST API + React frontend."
3 Coding (Implementation)	Convert design into actual code.	"Develop REST endpoints for adding, viewing, and deleting expenses."
4 Testing	Verify the product meets requirements and has no bugs.	"Check if adding an expense updates total automatically."
5 Rollout (Deployment & Maintenance)	Release the product to users and maintain it.	"Deploy on AWS, monitor errors, add new features."

◆ Characteristics:

- Linear and sequential
- Each phase has specific deliverables
- No going back once a phase is complete
- Suitable for projects with **clear, fixed requirements**

✅ Advantages:

- Easy to understand and manage
- Good for small, predictable projects
- Clear documentation at every step

❌ Disadvantages:

- Difficult to adapt to changes
- Late discovery of errors (during testing)
- Not ideal for complex, fast-changing environments

💡 Example:

Government or banking systems — where requirements rarely change and must be documented strictly.

⚡ Agile Model

Agile is a modern, flexible, and *iterative* approach to software development.

Instead of one long cycle, work is done in **short iterations** called "**sprints**" (usually 2–4 weeks).

Each sprint delivers a **small working piece** of the software that can be tested, used, and improved continuously.

◆ Structure:

📄 Copy

Plan → Design → Develop → Test → Review → Deploy → Repeat

Each iteration improves the product with feedback.

◆ Core Principles (from Agile Manifesto):

1. **Individuals and interactions** over processes and tools
2. **Working software** over documentation
3. **Customer collaboration** over contract negotiation
4. **Responding to change** over following a rigid plan

✖ Disadvantages:

- Requires **experienced teams**
- **Less documentation** may confuse newcomers
- Harder to predict total time and cost

✅ Advantages:

- **Very flexible** — adapts to changing requirements
- **Early and continuous delivery** of working software
- **Close collaboration** with clients and users
- **Faster feedback loops** → better quality

- **Motivated teams** — everyone sees progress each sprint

💡 Example:

Startups or tech companies (like Google, Spotify, or startups building AI apps) where requirements evolve quickly.

🚀 Why Agile is Preferred Today

Reason	Explanation
Faster delivery	Users get working features early
Adaptable to change	Requirements can change mid-project
Customer involvement	Continuous feedback ensures satisfaction
Improved quality	Bugs caught early every sprint
Team collaboration	Developers, testers, and managers work together daily
Reduced risk	Regular feedback prevents major failures late in the project

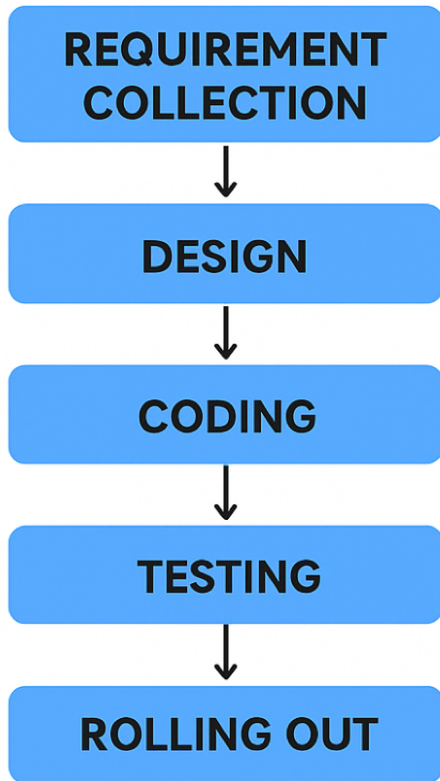
🧠 Summary Table

Feature	Waterfall	Agile
Approach	Sequential	Iterative
Change Handling	Difficult	Easy
Customer Involvement	Low	High
Working Software	Delivered at end	Delivered in every sprint
Flexibility	Rigid	Adaptive
Risk	Higher	Lower
Best for	Stable, fixed-scope projects	Evolving, dynamic projects

💬 Simple Analogy:

- **Waterfall** = Building a house → you can't move the kitchen after it's built.
- **Agile** = Building a Lego house → you can reassemble and improve it anytime.

WATERFALL



AGILE

